

UNIT 1, FOUNDATIONS OF AI AGENTS AND MICROSOFT FOUNDRY

LECTURE 1

What an Agent Actually Is

Where the boundary sits, why it is blurrier than anyone admits, and the four things a system needs before the word is honest.

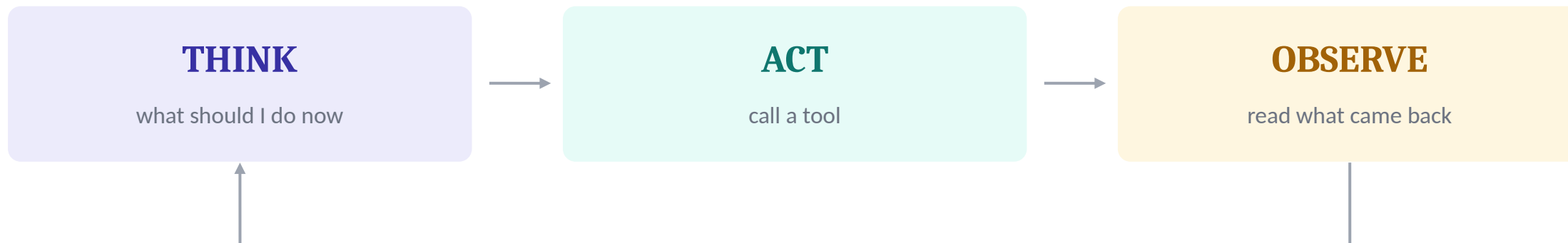
SECTION 00

Where we left off

Three words, and one question they leave open.

RECALL

Last time, in one picture



You saw this work. A goal went in, tools got used, and something finished. Nobody typed the intermediate steps.

THE QUESTION IT LEAVES OPEN

That was a demo, and demos are chosen to work. Today we ask the harder question. When somebody hands you a product and calls it an agent, how do you check whether that word is honest?

What you will be able to do by the end of today

- State the four things a system must have before the word agent is honest, and apply them to any product you are shown.
- Explain exactly what happens when a model uses a tool, including the part almost everybody gets wrong.
- Write a working agent from scratch, in about forty lines, with no framework at all.
- Break it on purpose, in two different ways, and fix both.
- Say confidently when an agent is the wrong choice, which is more often than the internet suggests.

HOLD ON TO THIS

The framework comes in Unit 3. Today you build it by hand, because a framework you cannot read underneath is a framework you cannot debug.

SECTION 01

The word is broken

Before we define it, we need to admit it currently means nothing.

Three products. All three are marketed as agents.

PRODUCT A

A support chat window

Answers from a help centre. Cannot look at your order. Cannot change anything. Escalates to a human when stuck.

PRODUCT B

An email sorter

Reads incoming mail, applies one of five labels, files it. Same five labels every time, in the same order.

PRODUCT C

A coding assistant

Given a bug report, reads the repo, edits files, runs tests, reads failures, edits again, opens a pull request.

Every vendor here uses the same word. Only one of these three is doing what we described last session.

THIRTY SECONDS, TALK TO THE PERSON NEXT TO YOU

Which one, and what exactly is your test for deciding? Do not say "it is smarter". Name the specific capability.

The layman version

LEVEL 0

The intern who asks

Comes to you before every single action. "Should I call the supplier now?" You are still doing the deciding.

LEVEL 1

The intern with a checklist

You wrote the steps. They follow them exactly, every time, in order. Reliable and completely inflexible.

LEVEL 2

The employee with a goal

You said "sort out the Rao Logistics problem". They decide what to do first, second, third. They come back when it is done.

THE AXIS THAT MATTERS

Notice what is changing down that list. Not intelligence. Not knowledge. What changes is who decides the next step. That single question is the whole definition.

Now the technical version

Same three levels, in the language you will meet in documentation and job descriptions.

LEVEL 0

Human in the loop

The system proposes, a person approves every action. Common in anything with legal or financial consequence.

LEVEL 1

Workflow

The sequence of steps is fixed by you, in code, at design time. The model fills in content but never chooses the path.

LEVEL 2

Agentic

The sequence of steps is decided by the model at run time, based on what it observes. You wrote the tools, not the order.

HOLD ON TO THIS

Level 1 versus Level 2 is a decision about who owns the control flow. In a workflow, you own it. In an agent, the model owns it and you own the boundaries.

The misconception to kill right now

WHAT PEOPLE SAY

Chatbot or agent, pick one

As if there were a line, and a product sits cleanly on one side of it.
Useful for marketing. Useless for engineering.

WHAT IS TRUE

Agency is a dial, not a switch

Real systems sit somewhere along the range, and the same product often sits at different points for different tasks.

Product A from two slides ago escalates to a human. That is a decision. A small one, from a menu of two, but a decision. It is not at zero.

SO THE USEFUL QUESTION IS NOT "IS IT AN AGENT"

The useful question is: how much of the control flow has been handed to the model, and what happens when it chooses badly? That is a question you can actually answer about a real system.

SECTION 02

The four requirements

Before the word is honest, all four have to be there.

Four things. Miss any one and it is something else.

01

A goal

Not a question. An outcome, with the steps left unspecified.

02

Tools

A way to affect or observe the world beyond its own text.

03

A loop with state

It runs more than once, and each run knows what the last one found.

04

A way to stop

A defined condition for being finished, and a budget for failing to be.

HOLD ON TO THIS

Three of these are obvious once stated. The fourth is the one nobody builds first, and it is the one that costs money.

Requirement 1, a goal, not a question

A QUESTION

"What is our worst supplier?"

A GOAL

"Find our worst supplier and draft the escalation email."

A question has one answer and one step. A goal has an outcome, and how many steps it takes is not your problem. That second part is the whole point. You are declining to specify the path.

THE MISCONCEPTION

A long, detailed question is still a question. "Open the supplier table, sort by delivery rate, take the lowest, then write an email" is not a goal. That is you doing the deciding and dictating it in English. If you already know the steps, write the steps in code. It will be cheaper, faster and more reliable.

Requirement 2, tools

A model without tools is a very well read person locked in a room with no phone.

WITHOUT TOOLS

It can only produce text

It knows a great deal, up to a date. It cannot see your database, cannot check today's price, cannot send anything, cannot verify a single claim it makes.

WITH TOOLS

It can reach the world

Query a table. Call an API. Read a file. Run a search. Every one of those is an ordinary function you wrote, that you decided to expose.

HOLD ON TO THIS

Tools are the difference between a system that describes work and a system that does it. In Unit 6 you will learn that they are also the entire attack surface.

A tool is just a function. Nothing special happens to it.

```
def get_room_availability(hotel: str, date: str) -> str:
    """How many rooms are free at a hotel on a date."""
    n = ROOMS.get((hotel, date))
    if n is None:
        return f"No record for {hotel} on {date}."
    return f"{hotel} on {date}: {n} rooms available."
```

That is a tool. There is no decorator, no registration with a vendor, no special base class. It is the function you would have written anyway.

ONE DESIGN RULE, STARTING NOW

One tool does one job. A tool called `handle_booking` that checks availability and computes price and sends confirmation will be called at the wrong moment and will fail in a way you cannot debug. We will watch that happen in Unit 2.

The part almost everybody gets wrong

The model never calls your function. It cannot. It has no ability to execute anything at all.

A language model produces text. That is the only thing it does, and giving it tools does not change that. What actually happens is a request and a reply, and your code sits in the middle of it.

WHY THIS MATTERS MORE THAN IT SOUNDS

If you believe the model executes things, then security, error handling and cost control all look like the vendor's problem. Once you understand that your loop is the thing doing the executing, you realise every one of those is yours.

What actually happens, in three moves

- 1 You ask** You send the conversation plus a list of tool descriptions. Plain text and JSON. Nothing executable.
- 2 It requests** The model replies with a message that says: I would like `get_room_availability` with these arguments. That is text. Nothing has run.
- 3 You execute** Your code looks up that name, checks it is allowed, calls the real function, and sends the result back as a new message.

the red box is the only step where anything actually runs, and it is your code

Step 3 is where security lives, where errors happen, and where your bill is generated.

The schema is the only thing the model ever sees

```
{
  "type": "function",
  "function": {
    "name": "get_room_availability",
    "description": "Get the number of rooms available at a
                    specific hotel on a specific date.",
    "parameters": {
      "type": "object",
      "properties": {
        "hotel": {"type": "string", "description": "..."},
        "date": {"type": "string", "description": "YYYY-MM-DD"}
      },
      "required": ["hotel", "date"]
    }
  }
}
```

The description is not documentation

It is the instruction the model uses to decide whether to pick this tool over another one. Vague description, wrong tool chosen, and the bug is in your English, not your Python.

The model cannot see

your function body, your variable names, your database, or what the function returns until after it has run.

Requirement 3, a loop that carries state

Running once is a function call. Running again, knowing what the first run found, is an agent.

THE LAYMAN VERSION

The notepad

Send someone to the godown three times. If they carry no notepad, the third trip starts exactly like the first. They will recount the same shelf and never finish.

THE TECHNICAL VERSION

The messages list

The notepad is a Python list. Every request, every tool result, every reply gets appended. You resend the whole list on every turn.

THE MISCONCEPTION

The model does not remember anything between calls. Not one word. Every call is the first call as far as it is concerned. The illusion of memory is entirely produced by you resending the transcript each time, and that is why long conversations cost more.

What the notepad looks like at step three

system You are a hotel booking assistant. You have tools for...

user Rooms and price at Taj Palace on 14 August?

assistant tool_calls: get_room_availability(Taj Palace, 2026-08-14)

tool Taj Palace on 2026-08-14: 3 rooms available.

assistant tool_calls: get_nightly_rate(Taj Palace)

tool Taj Palace: Rs 14500 per night.

All six of those go up on every single request. The model reads the whole thing again each time and decides what happens next from scratch.

Requirement 4, a way to stop

A shop with no closing time is not dedicated. It is broken.

EXIT 1

The goal is met

The model stops asking for tools and produces a final answer. This is the happy path and the only one most tutorials handle.

EXIT 2

The budget ran out

You capped the number of steps or the spend. It stops and says so honestly, rather than pretending it succeeded.

EXIT 3

No progress is being made

It has called the same tool with the same arguments three times. Something is wrong. Stop and escalate rather than continue.

THE MISCONCEPTION, AND IT IS AN EXPENSIVE ONE

People assume the model knows when it is finished. Sometimes it does. When it does not, it will keep requesting tools until something external stops it, and every one of those requests is billed. You will watch this happen live in about twenty minutes.

The four, one more time

01

A goal

Not a question. An outcome, with the steps left unspecified.

02

Tools

A way to affect or observe the world beyond its own text.

03

A loop with state

It runs more than once, and each run knows what the last one found.

04

A way to stop

A defined condition for being finished, and a budget for failing to be.

THE TEST YOU CAN NOW APPLY TO ANY PRODUCT

Go back to the three products from earlier. Product A has no tools and no loop. Product B has tools but the path is fixed by its author, so no real loop. Product C has all four. Only C earns the word.

SECTION 03

The anatomy

Six parts, and the syllabus words for each of them.

Six components. You will build every one of these this semester.

MODEL

The reasoning

Decides what happens next. Unit 1.

INSTRUCTIONS

The job description

Role, constraints, tone, what not to do. Unit 1.

TOOLS

The hands

Functions it may request. Units 2 and 3.

MEMORY

The notepad

What has happened so far. Unit 3.

LOOP

The engine

Runs the cycle, executes, feeds back. Unit 2.

GUARDRAILS

The boundaries

Budget, whitelist, filters, permissions. Units 5 and 6.

Today you build four of the six by hand. Memory and guardrails get one line each and a promise that we will do them properly later.

The syllabus words for the same thing

Your syllabus uses classical AI vocabulary. Here is the translation, so the exam paper and the code agree with each other.

PERCEPTION

How the agent takes in the world

For us: the tool results coming back, plus whatever is in the conversation. That is the entire sensory apparatus.

ACTION

How the agent changes the world

For us: the tool calls your loop executes. Reading a database is an action too, not just writing to one.

ENVIRONMENT

Whatever the agent can sense or affect

For us: your database, your APIs, your file system. The environment is exactly as large as the tools you exposed, and no larger.

HOLD ON TO THIS

The last one is a security statement disguised as a definition. The agent's reach is exactly the tool list. Nothing more. Remember that in Unit 6.

SECTION 04

Live build

Laptops open. Forty lines, no framework, and we break it twice.

What we are building

A hotel assistant that can answer a question needing two different lookups, without you telling it which order to do them in.

"Can I get a room at Taj Palace on 14 August, and what would it cost?"

Two tools exist: one for availability, one for rates. Nowhere in our code will we say "check availability first". The model works that out, and we will watch it do so.

OPEN THIS NOW

notebooks/u1/l1_first_agent.ipynb in the course repository. Run the first cell. If it prints your name, you are set. If it does not, put your hand up now rather than in ten minutes.

Step 1, the tools. Ordinary functions.

```
ROOMS = {("Taj Palace", "2026-08-14"): 3,  
         ("Radisson Blu", "2026-08-14"): 11}  
RATES = {"Taj Palace": 14500, "Radisson Blu": 6200}  
  
def get_room_availability(hotel: str, date: str) -> str:  
    n = ROOMS.get((hotel, date))  
    if n is None:  
        return f"No record for {hotel} on {date}."  
    return f"{hotel} on {date}: {n} rooms available."  
  
def get_nightly_rate(hotel: str) -> str:  
    return f"{hotel}: Rs {RATES[hotel]} per night."
```

NOTICE

Both return a string, not a dict. The result goes back to the model as text, so returning something readable saves you a serialisation step and makes your traces far easier to read at 2am.

Step 2, the whitelist and the schema

```
REGISTRY = {  
    "get_room_availability": get_room_availability,  
    "get_nightly_rate": get_nightly_rate,  
}
```

The registry is not a convenience. It is the whitelist. If a name is not a key in this dict, it does not run, no matter what the model asks for. We will test that claim in about ten minutes by making the model ask for something that does not exist.

THE SCHEMA GOES NEXT TO IT

TOOL_SCHEMA is the JSON description from earlier, one entry per function. Two lists that must stay in step: one your code uses, one the model reads. Getting them out of step is a classic bug, and Unit 3 shows you how frameworks generate both from one source.

Step 3, the loop. This is the whole thing.

```
messages = [{"role": "system", "content": SYSTEM},
             {"role": "user", "content": goal}]

for step in range(1, max_steps + 1):
    response = client.chat.completions.create(
        model=model, messages=messages, tools=TOOL_SCHEMA)
    message = response.choices[0].message
    messages.append(message.model_dump(exclude_none=True))

    if not message.tool_calls:
        return message.content

    for call in message.tool_calls:
        args = json.loads(call.function.arguments)
        result = REGISTRY[call.function.name](**args)
        messages.append({"role": "tool",
                        "tool_call_id": call.id,
                        "content": result})
```

Three things to notice

The for loop is the budget guard.

The assistant turn is appended whether or not it asked for a tool.

The tool result is tied to call.id, so the model knows which request it answers.

→ this line is the
only place code runs

Run it. Read the trace.

```
>>> run_agent(client, MODEL,
...           "Can I get a room at Taj Palace on 2026-08-14,
...           and what would it cost?")

[step 1] get_room_availability({'hotel': 'Taj Palace',
                               'date': '2026-08-14'})
        -> Taj Palace on 2026-08-14: 3 rooms available.
[step 2] get_nightly_rate({'hotel': 'Taj Palace'})
        -> Taj Palace: Rs 14500 per night.
[step 3] done

'Yes, Taj Palace has 3 rooms available on 14 August
at Rs 14500 per night.'
```

THE THING WORTH SEEING

Nowhere did we say "availability first, then price". The model chose that order. Change the question to ask only about price and step 1 disappears. That is the control flow living in the model rather than in your code, which is exactly the Level 2 definition from the start of the lecture.

FAILURE ONE

Now break it. On purpose.

Delete the max_steps guard, and give it a goal that none of its tools can satisfy.

```
while True:                                # the guard is gone
    ...

run_agent(client, MODEL,
    "Book me a room at the Oberoi in Shimla for tonight")
```

There is no Oberoi in the data and there is no booking tool. Watch what it does rather than what you expect it to do.

WHAT ACTUALLY HAPPENS

It calls get_room_availability. Gets "no record". Tries a different date. Gets "no record". Tries the other hotel. Tries the first hotel again. Every attempt is a billed API call, and it is being helpful, not broken. This is the single most common way people get a surprise invoice.

The fix is one line, and it is not clever

```
for step in range(1, max_steps + 1):  
    ...  
  
return (f"Stopped after {max_steps} steps without reaching  
        f"a final answer.")
```

WHAT IT DOES

Bounds the cost

The worst case is now arithmetic you can do in advance instead of a number you discover on an invoice.

WHAT IT DOES NOT DO

Fix the underlying problem

The agent still cannot answer. The guard turns an unbounded failure into a bounded one. That is worth a lot and it is not a solution.

HOLD ON TO THIS

Returning an honest failure message beats returning a confident guess. Unit 5 is largely about that distinction.

FAILURE TWO

Break it again. Differently.

This time we do not touch the code. We just ask for something the model would like a tool for, and does not have.

```
run_agent(client, MODEL,  
    "Check the Taj Palace and email the result to the manager")
```

```
[step 1] get_room_availability(...) -> 3 rooms available.
```

```
[step 2] send_email({...})  
    -> Error: no tool named 'send_email'.  
        Available: ['get_room_availability',  
                    'get_nightly_rate']
```

```
[step 3] done
```

```
'Taj Palace has 3 rooms. I do not have an email tool,  
so I could not send that.'
```

Why that did not blow up

```
if name in REGISTRY:
    result = REGISTRY[name](**args)
else:
    result = f"Error: no tool named '{name}'."
```

THE NAIVE VERSION

REGISTRY[name](args)**

A `KeyError`, an unhandled exception, and in a web service a 500 to your user. Caused by the model saying a word.

THE DEFENDED VERSION

Check, then tell it

The error goes back as an observation. The model reads it, adjusts, and answers honestly. The loop survives.

THE PRINCIPLE, WHICH RETURNS IN UNIT 6

Anything the model produces is untrusted input. A tool name is a string that came from a language model, and you validate it exactly as carefully as you would validate a form field from the open internet.

What you just wrote, and what a framework adds

YOURS, TODAY

About forty lines

Loop, whitelist, budget, tool results fed back, honest failure. Every requirement from section two, visible on one screen.

WHAT SEMANTIC KERNEL ADDS

Structure, not magic

Schema generated from your type hints, retries, streaming, memory backends, tracing, and a shape that survives more than two tools.

WHAT IT DOES NOT ADD

A different loop

Underneath, it is the same think, act, observe cycle you just typed. Every framework in this course is doing what is on your screen right now.

HOLD ON TO THIS

This is why we built it by hand first. When something misbehaves in Unit 3, you will know what layer to look at, because you have already written that layer.

SECTION 05

When not to use one

The most professional thing you will learn today.

Three shapes. Only one is an agent.

WORKFLOW

You wrote the path

Step one, then two, then three. Every time.
The model fills in content but never chooses direction.

GOOD FOR

Extracting fields from ten thousand invoices.

ROUTER

One decision, then a path

The model picks a branch from a fixed set,
and that branch is ordinary code.

GOOD FOR

Sorting support tickets into five queues.

AGENT

The model picks the path

Number of steps unknown in advance. Order
decided from what it observes.

GOOD FOR

Investigating why a build broke.

Most production systems that get called agents are actually the first or the second, and they are better for it.

The test, and it is one question

Can you write down the steps in advance? If yes, write them. You do not need an agent.

An agent is the right tool when the number of steps genuinely depends on what you find partway through. Debugging. Research. Investigation. Anything where step three is unknowable until step two returns.

WHAT AN AGENT COSTS YOU FOR THAT FLEXIBILITY

Several model calls instead of one, so more money and more latency. Non deterministic behaviour, so testing gets hard. A larger surface for things to go wrong. You are paying all of that specifically to buy an unknown step count. If you do not need that, you are paying for nothing.

The misconception, stated plainly

WHAT THE INTERNET SAYS

Agents are the advanced option

So a workflow must be the beginner version, and using one means you did not understand the material.

WHAT ENGINEERS SAY

Agents are the expensive option

You reach for one when the problem shape demands it. Choosing a workflow when a workflow fits is the senior answer, not the junior one.

HOLD ON TO THIS

In your practical viva, "why is this an agent and not a script" is a real question with real marks attached. "Because the course is called agentic AI" is not an answer.

Unit 2 spends a full block on exactly this decision.

Hold on to this

An agent is a loop that decides its own next step, using tools you gave it, until a condition you defined is met.

Every word in that sentence is load bearing. Loop, decides, tools you gave it, condition you defined. Remove any one and you have something else, and you should call it something else.

The model never executes anything

Your loop does. That is where security, errors and cost live.

The model remembers nothing

You resend the transcript every turn. That is what memory is.

A tool description is an instruction

Vague description, wrong tool, and the bug is in your English.

Termination is a feature

Not an afterthought. It is requirement four for a reason.

Before next session

DO

Practical 1

Environment setup across all four lanes. The verification script has to print "You are ready." Everything after this depends on it.

DO

Extend the agent

Add a third tool of your own to today's notebook. Something the hotel assistant would plausibly need. Commit it.

THINK

Find a bad example

Find one product online marketed as an AI agent. Apply the four requirements. Be ready to say which ones it fails.

ON THE THIRD ONE

Most of you will find a product that fails at least two of the four. That is not cynicism, it is the exercise working. Bring the specific product, not a general complaint.

Next session

Unit 1, Lecture 2

Intelligent Agent Architectures

Today we built one architecture without naming it. Next time we name the whole family. Reactive, goal based, utility based and learning agents, what each one is actually good for, and which of them your forty lines already is.

COME WITH

Practical 1 done, your third tool committed, and one badly marketed agent product to pull apart.